

Vehiculo de Transporte de Tubos de Gases para Soldadura

Especificacion funcional del modulo de gestion de vehiculos transportadores de tubos de gases industriales (oxigeno, acetileno, argon, CO2) para una aplicacion Python/React. Incluye modelo de datos, endpoints API, componentes de interfaz y logica de compartimentos por tamano de tubo.

STACK TECNOLÓGICO

Python (FastAPI/Django) + React + PostgreSQL

FECHA

Junio 2026

1. Arquitectura General del Modulo

El modulo **GasTubeTransport** se integra dentro de una aplicacion existente con stack Python en el backend (preferentemente FastAPI o Django REST Framework) y React en el frontend. La comunicacion entre capas se realiza mediante una API RESTful JSON, mientras que la persistencia de datos se gestiona a traves de PostgreSQL con ORM (SQLAlchemy o Django ORM). El modulo es autocontenido y puede importarse como un paquete independiente dentro del proyecto principal, exponiendo sus propios modelos, serializers, vistas y componentes de interfaz sin acoplarse a logica de negocio ajena.

La arquitectura sigue un patron de capas clasico: los componentes React consumen los endpoints del backend a traves de un cliente HTTP (axios o fetch), el backend valida las reglas de negocio y persiste los cambios en la base de datos. Cada entidad principal (Vehicle, TubeType, Stock, LoadRecord) cuenta con su propio conjunto de endpoints CRUD, mas operaciones especificas como carga/descarga de tubos, consultas de inventario y alertas de stock minimo. Esta separacion permite que un agente de IA pueda generar cada capa de forma independiente, manteniendo contratos claros entre el frontend y el backend.

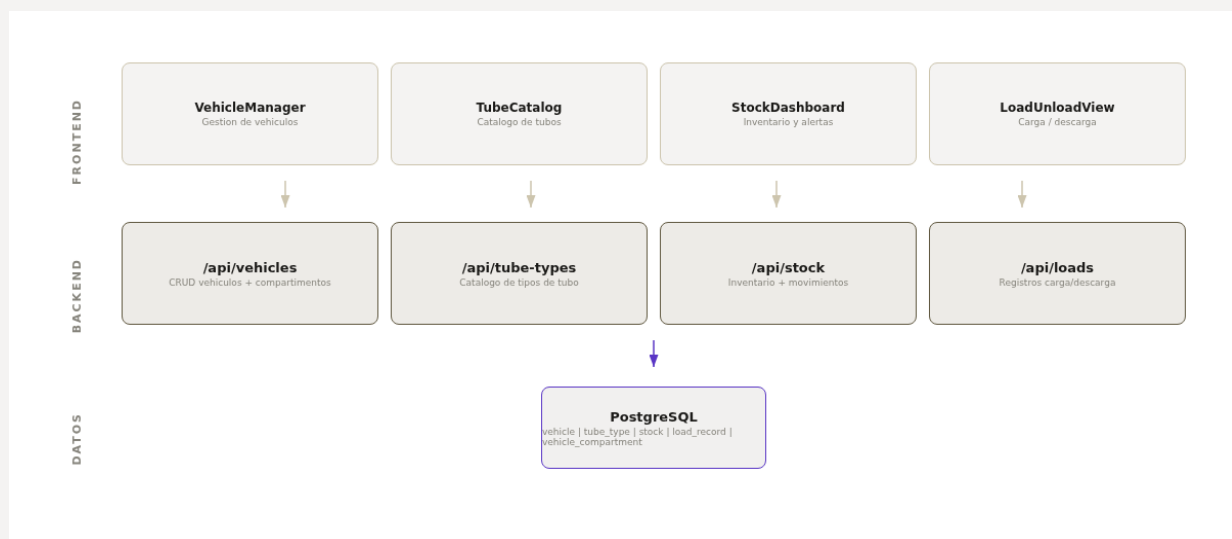


Figura 1. Arquitectura de tres capas del modulo GasTubeTransport: componentes React, endpoints Python y esquema PostgreSQL.

Estructura de Carpetas Sugerida

Para que el agente pueda generar el codigo de forma coherente, se recomienda la siguiente estructura de directorios. El backend organiza los modelos, serializers, vistas y servicios en subcarpetas dedicadas, mientras que el frontend agrupa componentes, hooks y servicios de llamada a la API. Esta convencion facilita la navegacion y el mantenimiento del modulo.

```

backend/
  gas_transport/
    models/      # vehicle.py, tube_type.py, stock.py, load_record.py
    serializers/ # esquemas de validacion Pydantic
    views/       # endpoints API
    services/    # logica de negocio (carga/descarga)
    routes.py    # registro de rutas
frontend/src/
  modules/gas-transport/
    components/  # VehicleManager, TubeCatalog, StockDashboard, LoadUnload
    hooks/       # useVehicles, useStock, useTubeTypes
    services/    # api.ts (cliente HTTP)
    types/       # interfaces TypeScript
  
```

2. Modelo de Datos

El esquema relacional se compone de cinco entidades principales que capturan la totalidad del dominio: vehículos con sus compartimentos, tipos de tubo con especificaciones técnicas, inventario de stock con alertas, registros de carga y descarga con trazabilidad, y operadores que ejecutan los movimientos. Cada relación está diseñada para soportar consultas eficientes y mantener la integridad referencial. Los campos UUID como claves primarias garantizan unicidad global, mientras que los enums restringen los valores permitidos en campos de estado y tipo de movimiento.

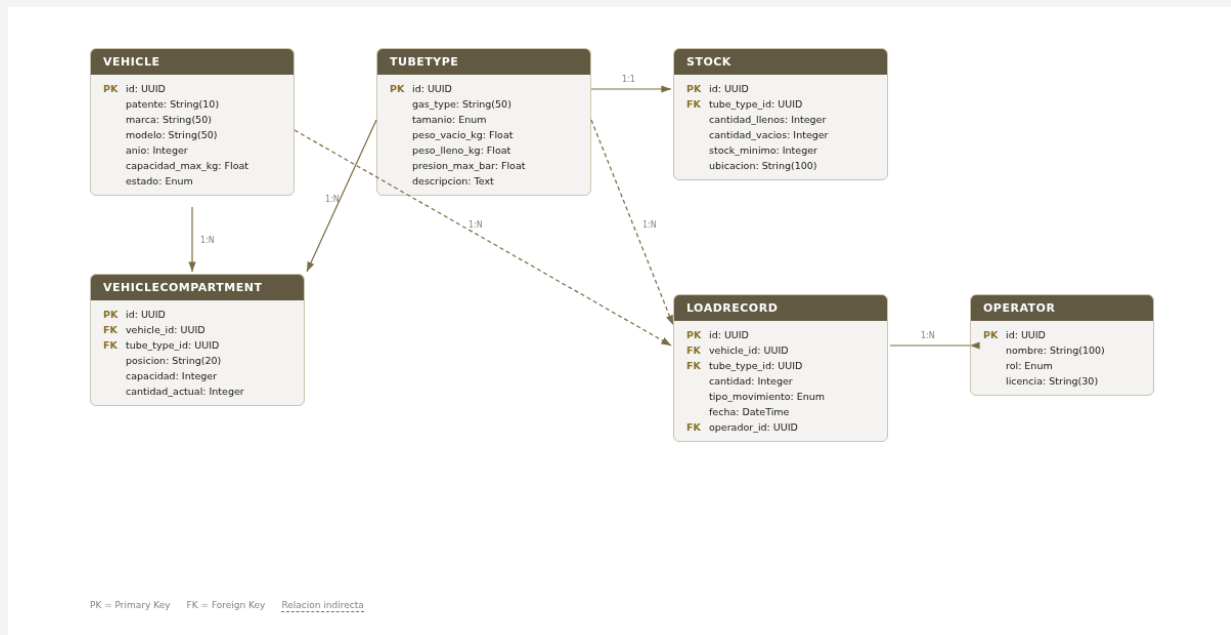


Figura 2. Diagrama Entidad-Relacion del módulo. Las líneas discontinuas indican relaciones indirectas a través de entidades intermedias.

Descripción de Entidades

Vehicle: Representa cada vehículo de transporte con su patente única, marca, modelo, año de fabricación y capacidad máxima de carga en kilogramos. El campo **estado** es un enum con valores: *disponible*, *en_ruta*, *en_mantenimiento*, *fuera_de_servicio*. La capacidad máxima se utiliza para validar que la suma de pesos de los tubos cargados no la supere en ningún momento, calculada dinámicamente a partir de los compartimentos ocupados y los pesos de los tipos de tubo asignados a cada uno.

VehicleCompartment: Cada vehículo se divide en compartimentos, cada uno asociado a un tipo de tubo específico (tamaño). Esto permite que el sistema valide que solo se carguen tubos del tamaño correcto en cada compartimento. Los campos **posición** (ej: "A1", "B2") y **capacidad** definen la ubicación física y el límite de unidades, mientras que **cantidad_actual** refleja el estado en tiempo real del compartimento.

TubeType: Catálogo maestro de tipos de tubo de gas. Incluye el tipo de gas (oxígeno, acetileno, argón, CO2, mezcla MIG), el tamaño del tubo (pequeño 10L, mediano 20L, grande 50L), pesos vacío y lleno, presión máxima de trabajo en bar y una descripción libre. Este catálogo es referenciado por compartimentos, stock y registros de carga, garantizando consistencia en toda la aplicación.

Stock: Registra el inventario disponible en depósito para cada tipo de tubo, diferenciando entre tubos llenos y vacíos. El campo **stock_minimo** permite configurar alertas automáticas cuando la cantidad de tubos llenos cae por debajo del umbral, notificando al operador para gestionar la reposición oportuna. La ubicación indica el depósito o planta física donde se encuentran los tubos.

LoadRecord: Cada operación de carga o descarga de tubos en un vehículo genera un registro de auditoría. El campo **tipo_movimiento** es un enum: *carga*, *descarga*. Se registra la fecha, hora, el operador responsable y la cantidad de tubos movidos. Estos registros permiten reconstruir la historia completa de movimientos de cada vehículo y cada tipo de tubo, esencial para trazabilidad y conciliación de inventarios.

3. Modulo del Vehiculo y Catalogo de Tubos

Cada vehiculo de transporte se configura con un conjunto de compartimentos que definen su capacidad para distintos tamanos de tubo. La configuracion de compartimentos es flexible y puede variar entre vehiculos, permitiendo adaptar la flota a diferentes necesidades de distribucion. El sistema valida automaticamente que las operaciones de carga respeten tanto la capacidad por compartimento (numero de tubos) como la capacidad total del vehiculo (peso en kilogramos), calculada como la suma de (cantidad_actual * peso_lleno_kg) de cada compartimento ocupado.



Figura 3. Vista superior esquematica del vehiculo con seis compartimentos distribuidos en dos filas de tres, clasificados por tamaño de tubo: grande (50L), mediano (20L) y pequeño (10L).

Catalogo de Tubos de Gases

El catalogo define los tipos de gas y tamanos de tubo disponibles en el sistema. Cada tipo de gas tiene propiedades fisicas especificas (peso, presion maxima) que son fundamentales para el calculo de capacidad de carga del vehiculo. A continuacion se presenta la tabla de referencia con los tipos de gas y tamanos tipicos que el agente debe implementar como datos iniciales (seed data) del sistema.

Tipo de Gas	Tamano	Peso Vacio (kg)	Peso Lleno (kg)	Presion Max (bar)
Oxigeno (O2)	Pequeno 10L	5.5	7.0	200
Oxigeno (O2)	Mediano 20L	18.0	25.0	200
Oxigeno (O2)	Grande 50L	55.0	75.0	200
Acetileno (C2H2)	Mediano 20L	22.0	30.0	15
Acetileno (C2H2)	Grande 50L	65.0	85.0	15
Argon (Ar)	Mediano 20L	20.0	28.0	200
Argon (Ar)	Grande 50L	58.0	78.0	200
CO2	Mediano 20L	22.0	35.0	57
Mezcla MIG (Ar/CO2)	Pequeno 10L	6.0	8.5	200

Tabla 1. Especificaciones tecnicas de referencia para los tipos de tubo del catalogo. El agente debe utilizar estos valores como datos semilla al inicializar la base de datos.

Estados del Vehiculo

El campo **estado** del vehiculo sigue una maquina de estados finita que restringe las transiciones permitidas. Desde *disponible* se puede pasar a *en_ruta* (al iniciar una entrega) o a *en_mantenimiento*. Desde *en_ruta* solo se puede volver a *disponible* al completar la ruta. Desde *en_mantenimiento* se vuelve a *disponible* al finalizar las reparaciones. El estado *fuera_de_servicio* es terminal y requiere intervencion administrativa para reactivarse. El backend debe validar cada transicion y retornar HTTP 409 (Conflict) si se intenta una transicion invalida.

4. Gestion de Stock y Endpoints API

La gestion de stock controla el inventario de tubos llenos y vacios en cada ubicacion de deposito. Cuando se carga un vehiculo, se descuenta del stock de tubos llenos y se incrementa el stock de vacios en la ubicacion de origen. Cuando se descarga, se opera de forma inversa. El sistema genera alertas cuando la cantidad de tubos llenos de un tipo determinado cae por debajo del umbral configurado en **stock_minimo**, permitiendo planificar la reposicion antes de que se agote el suministro. Estas alertas pueden exponerse tanto en el dashboard de React como via notificaciones push o email.

Endpoints API (Python Backend)

Metodo	Endpoint	Descripcion
GET	/api/vehicles	Listar vehiculos con filtros (estado, patente). Incluye compartimentos y conteo de tubos.
POST	/api/vehicles	Crear vehiculo con su configuracion de compartimentos.
GET	/api/vehicles/{id}	Detalle del vehiculo con compartimentos, estado de carga y peso actual.
PATCH	/api/vehicles/{id}/estado	Transicionar estado del vehiculo (valida maquina de estados).
GET	/api/tube-types	Listar tipos de tubo del catalogo con filtros (gas, tamano).
POST	/api/tube-types	Crear nuevo tipo de tubo en el catalogo.
GET	/api/stock	Consultar inventario actual por ubicacion y tipo de tubo.
GET	/api/stock/alerts	Obtener lista de tipos de tubo por debajo del stock minimo.
POST	/api/loads	Registrar operacion de carga/descarga. Actualiza stock y compartimentos.
GET	/api/loads	Historial de movimientos con filtros (vehiculo, fecha, tipo).

Tabla 2. Endpoints REST del modulo. Todos retornan JSON; errores usan formato estandar {detail, code}. El endpoint POST /api/loads es transaccional: actualiza stock, compartimento y crea el registro en una unica transaccion de base de datos.

Logica de Carga/Descarga

La operacion de carga es el flujo mas critico del modulo. Cuando se ejecuta un POST a /api/loads con tipo_movimiento=carga, el backend debe: (1) verificar que el vehiculo este en estado *disponible*, (2) buscar el compartimento del vehiculo asociado al tipo de tubo, (3) validar que haya espacio suficiente en el compartimento ($\text{capacidad} - \text{cantidad_actual} \geq \text{cantidad solicitada}$), (4) validar que el peso total no supere *capacidad_max_kg* del vehiculo, (5) descontar del stock de tubos llenos y aumentar vacios en la ubicacion de origen, (6) incrementar *cantidad_actual* del compartimento, y (7) crear el registro LoadRecord. Todo esto debe ejecutarse dentro de una transaccion atomica de base de datos para garantizar la consistencia del sistema ante fallos parciales.

5. Componentes React e Interfaz de Usuario

El frontend se estructura como un modulo independiente dentro de la aplicacion React, con componentes organizados por responsabilidad funcional. Cada componente consume los endpoints del backend a traves de hooks personalizados (useVehicles, useStock, useTubeTypes, useLoadRecords) que encapsulan la logica de llamada a la API, manejo de estados de carga, errores y cacheo de datos. La interfaz debe ser responsiva y accesible, siguiendo las convenciones de diseno de la aplicacion existente (shadcn/ui, Material UI, o similar).

Componente	Ruta Sugerida	Descripcion y Props Clave
VehicleManager	modules/gas-transport/component s/	Lista de vehiculos con filtros. Props: onEdit, onSelect. Muestra patente, estado (badge de color), carga actual y capacidad.

VehicleDetail	modules/gas-transport/components/	Detalle del vehículo con vista de compartimentos. Muestra grilla visual de compartimentos con ocupación.
TubeCatalog	modules/gas-transport/components/	Tabla editable del catálogo de tipos de tubo. CRUD completo con validación de campos técnicos.
StockDashboard	modules/gas-transport/components/	Dashboard de inventario con tarjetas por tipo de gas. Incluye indicadores de alerta (stock bajo).
LoadUnloadForm	modules/gas-transport/components/	Formulario para registrar cargas/descargas. Selecciona vehículo, tipo de tubo, cantidad y valida en tiempo real.
LoadHistory	modules/gas-transport/components/	Tabla historial de movimientos con filtros por fecha, vehículo y tipo de movimiento. Paginación server-side.

Tabla 3. Componentes React del módulo con rutas sugeridas y descripción funcional. El agente debe generar cada componente como un archivo .tsx independiente con sus correspondientes pruebas unitarias.

Tipos TypeScript (interfaces)

El agente debe generar los siguientes tipos TypeScript como contratos de interfaz entre el frontend y el backend. Estos tipos deben reflejar fielmente el modelo de datos del capítulo 2, garantizando consistencia en ambas capas de la aplicación. Se recomienda colocarlos en el archivo types/index.ts del módulo.

```
interface Vehicle {
  id: string; patente: string; marca: string; modelo: string;
  anio: number; capacidad_max_kg: number; estado: VehicleEstado;
  compartimentos: VehicleCompartment[];
}
interface TubeType {
  id: string; gas_type: string; tamaño: TubeTamano;
  peso_vacio_kg: number; peso_lleno_kg: number;
  presion_max_bar: number; descripcion: string;
}
type VehicleEstado = "disponible" | "en_ruta" | "en_mantenimiento" | "fuera_de_servicio";
type TubeTamano = "pequeno_10L" | "mediano_20L" | "grande_50L";
type TipoMovimiento = "carga" | "descarga";
```